Empirical Software Engineering (2025) 30:121
https://doi.org/10.1007/s10664-025-10679-1

Less-activated visual networks to enhance friendly following ability of lightweight robots

Jianxin Xue[1] · Husheng Chen[1] · Yaohua Hu[1] · Sichen Hua[1] · Minyu Chen[2] · Ling-I Wu[2] · Xi Chang[1]

Accepted: 20 May 2025 / Published online: 5 June 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Lightweight robots engaged in real-time following tasks require low-latency object detection to provide effective companionship services. However, these robots are often CPU-edge devices with limited memory, leading to frequent memory access in detection architectures like the YOLO series, causing significant delays. To address this challenge, we introduce LALCNet, a less-activated visual network derived from the CPU convolutional neural network PP-LCNet. LALCNet removes unnecessary activation functions and incorporates dual reduction strategies to optimize detection speed, while minimizing accuracy loss. By replacing the backbone network in the YOLO series, LALCNet processes entire images more efficiently in memory-constrained environments. Our comparative experiments show that LALCNet boosts detection speed on CPU-edge devices by 64%, 2.66X, and 1.93X compared to YOLOv5, YOLOv8, and YOLOv10, respectively. Despite these significant speed improvements, the decline in mean average precision (mAP) at a threshold of 0.5 is minimal. LALCNet emphasizes speed-first optimization while maintaining an acceptable level of accuracy, making it ideal for real-time robot following tasks. It enhances the user experience by enabling faster, more responsive interactions without compromising detection quality.

Keywords Real-Time Detection · Activation · Detection Latency · CPU-edge Robots

1 Introduction

Lightweight robots are often engaged in real-time following tasks to provide companionship services, acting as caregivers for patients (Tasaki et al. 2015) and nannies for babies or children (Islam et al. 2019). Their effectiveness in following capabilities is heavily reliant on

Communicated by: Zhang Tao, Xin xia

This article belongs to the Topical Collection: *Special Issue on Internetware 2024*

✉ Jianxin Xue
jxxue@sspu.edu.cn

[1] School of Computer and Information Engineering, Institute for Artificial Intelligence, Shanghai Polytechnic University, Shanghai, China

[2] Shanghai Jiao Tong University, Shanghai, China

low-latency object detection architectures, which enable friendly interactions with the person they are caring for.

To facilitate friendly following behavior towards the person being cared for, a low-latency object detection architecture embedded in the lightweight robot is crucial. For example, as illustrated in Fig. 1, the robot experiences latency in detecting the target (the child). By the time the robot identifies the child, the child may have already moved a certain distance. If the deviation is within the adjustable range (shown in Fig. 1 (a)), or as small as possible, the robot is likely to successfully complete the target-following task. However, if the detection time is too long, or the deviation exceeds the adjustable range (shown in Fig. 1 (b)), the robot is highly likely to lose track of the target. The reason lies in that it becomes increasingly difficult for predicting the target's movement trajectory. Therefore, improving identification speed, essentially minimizing detection latency, is vital for maintaining positive user experiences during interactions.

Minimizing detection latency presents significant challenges for lightweight robots. Firstly, these robots are typically CPU-edge devices with limited memory capacity. Secondly, object detection involves complex image processing, and latency encompasses the entire duration from the input of an image to the output of the result. This includes input image preprocessing time, inference time, and memory access time. Advanced object detectors, such as YOLOv8, also encounter difficulties on these constrained devices. YOLOv8 attempts to accelerate inference by partitioning the input images into multiple segments; however, this approach introduces substantial memory access demands. Each segment's intermediate results must be stored individually, leading to frequent memory access. This can exacerbate memory access latency issues.

In this study, we propose a detection architecture called LALCNet-YOLOs. The core idea is to utilize a novel neural network, termed **L**ess-**A**ctivated PP-**LCNet** (LALCNet), as a replacement for the backbone network of YOLO. We select YOLOv5s (Wu et al. 2021), YOLOv8s, and YOLOv10s as our detection infra-structure due to its advantage of not requiring image segmentation. LALCNet is derived from the CPU convolutional neural network PP-LCNet and involves the removal of certain unnecessary activation functions. Furthermore,

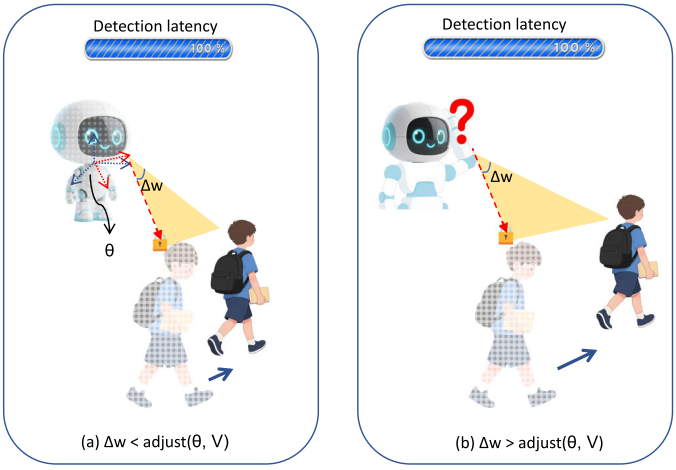


Fig. 1 Friendly and Unfriendly Following Scenarios of CPU-Edge Robots: Deviation between Locked Target Perspective and Actual Target Perspective (δw), Robot's Neck Degrees of Freedom θ, and Visible Scope (V) on Adjustable Scope (adjustScope(θ, V))

once the decision is made to remove these activation functions, we employ dual reduction strategies to further minimal detection latency and acceptable accuracy loss.

Experimental results on publicly available datasets demonstrate a significant reduction in latency compared to the native YOLOv5. We set detection latency as identification speed metric and mean Average Precision (mAP) at 0.5 Intersection over Union (IoU) threshold mAP(0.5) as accuracy metric. LALCNet-YOLOv5 shows remarkable speed improvements, outperforming native YOLOv5 by up to 62.8% on the COCO-FOOT-AND-LEG dataset and 81.3% on the PASCAL VOC dataset. In terms of detection accuracy, LALCNet-YOLOv5 incurs a marginal loss of 0.077 and 0.165 for these datasets, respectively. These notable speed enhancements, combined with a minor decrease in accuracy, demonstrates LALCNet-YOLOv5 aligned with the requirement of maintaining positive user experiences during interactions.

The contributions of this paper are summarized as follows:

- We design a serialized observation way to identify unnecessary activation computations based on examining the feature map generation process and assessing the impact of removing certain calculations.
- We propose a less-activated visual network called LALCNet, which remove unnecessary activation functions and adopt dual reduction strategies to minimize detection latency while maintaining an acceptable accuracy loss.
- We propose a novel object detection architecture, LALCNet-YOLOv5, which replaces the backbone network of YOLOv5 with LALCNet. This modification significantly enhances following ability of lightweight robots comparative with original YOLOv5.

2 Background

2.1 YOLO

The mainstream real-time object detector, YOLO series, specifically YOLOv5, YOLOv8 and YOLOv10 (Hussain 2023), are the same of architecture in Fig. 2. The *backbone* component is used to extract the features of input images. The *nest* component enhance the representation power of feature model to better handle targets of different scales and complexities. The **head** component, comprising multiple detection layers, facilitates multi-scale detection capabilities, enabling the model to discern targets of varying sizes.

The backbone network of the YOLO architectures is CSPDarknet. In YOLOv8 and YOLOv10, the CSPDarknet is enhanced by integrating a C2f structure, which replaces the original C3 structure to improve the model's robustness and overall performance. The C2f module incorporates a split operation that divides the feature map into multiple segments. This division facilitates parallel processing and more effective feature extraction. However, it also introduces significant memory access demands, as each intermediate output must be stored separately. This can pose challenges for memory-constrained devices, potentially affecting the model's usability. Consequently, YOLOv5 emerges as an optimal choice for real-time object detection in lightweight CPU-edge environments, where efficient memory usage is crucial.

2.2 PP-LCNet

The PP-LCNet is a lightweight CPU convolutional neural network (Cui et al. 2021), which employs a series of depthwise separable convolutional blocks. Every block includes two

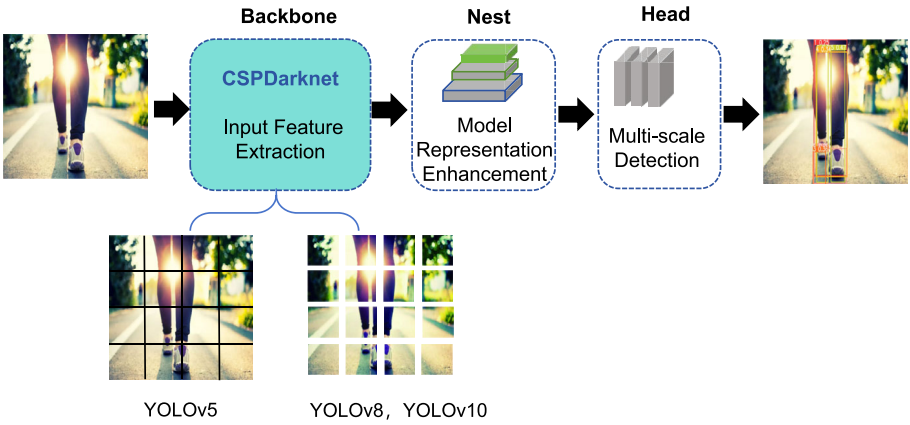


Fig. 2 Architecture of YOLO series

modulars: a convolutional modular (Conv) and a squeeze-and-excitation modular (SE). A $Conv$ modular consists of two convolution steps: 1) a depthwise convolution step (DW) and 2) a pointwise convolution step (PW).The SE module integrates the channel features following the two convolution steps.

The DW operates on each input channel independently. For an input feature map $X \in R^{H\times W\times C_{in}}$, where H is the height, W is the width, and C_{in} is the number of input channels, the DW with kernel $K_{dw} \in R^{K\times K\times 1}$, where K is the size of the convolution kernel, is performed as:

$$Y_{dw} = X^{H\times W\times C_{in}} * DWConv_{k_{dw}\times k_{dw}} \tag{1}$$

The output Y_{dw} is of shape $H' \times W' \times C_{in}$, where H' and W' are the height and width of the output after the convolution operation. During the process of feature extraction, the convolution kernels k_{dw} for the depthwise step of the last 7 blocks are 5×5, while those for the front blocks are all 3×3.

The PW convolution is applied to combine the inter-channel dependencies after the depthwise convolution. Given the output of the depthwise convolution $Y_{dw} \in R^{H'\times W'\times C_{in}}$, the pointwise convolution uses a kernel k_{pw} to produce the final output Y:

$$Y = Y_{dw}^{H'\times W\times C_{in}} * PWConv_{k_{pw}\times k_{pw}} \tag{2}$$

where k_{pw} is 1×1.

The $H - swish$ activation function is applied after both the depthwise and pointwise convolutions to introduce nonlinearity, enhancing the network's expressive capacity.

$$Y_{dw}^{act} = H - swish(Y_{dw}) \tag{3}$$

$$Y^{act} = H - swish(Y) \tag{4}$$

Based on our investigation, it is evident that the backbone network accounts for an average detection latency of 62% in YOLOv5, significantly surpassing the combined latency of the other segments. Hence, the decision to replace the backbone network with PP-LCNet while retaining other components emerges as a rational detection architecture for lightweight robots.

3 Motivation

To improve identification speed without excessive accuracy losses, it is a key point to discover unnecessary computations in the backbone network, i.e., PP-LCNet. Based on the analysis of PP-LCNet's computations in Section 2.2 , we observe the behavior of the activation function by analyzing the feature map sequence using PP-LCNet. We examine the four cases related to the activation function, as illustrated in Fig. 3.

3.1 Observation 1

Observing Fig. 3 (a) and (b), which respectively represent the cases with and without activation after DW, we can clearly see the differences in the feature map details. Figure (a) presents a sharper and more detailed representation of the feature map, showcasing well-defined contours and structures that provide clear spatial information. In contrast, Figure (b)

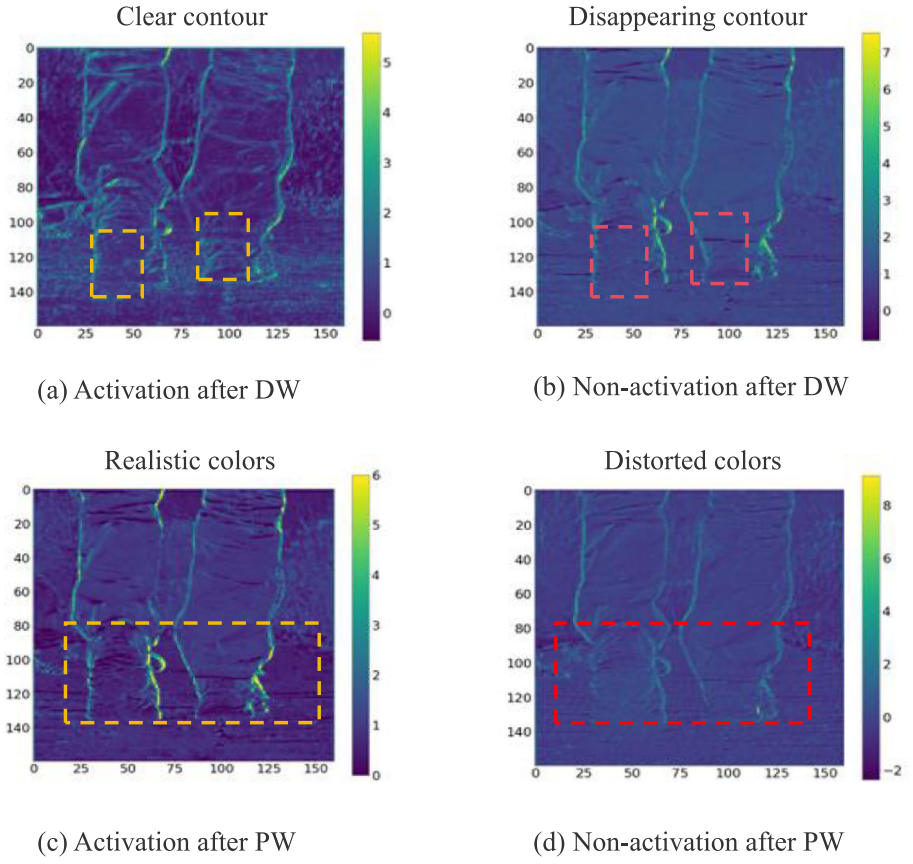


Fig. 3 The feature maps are shown in four distinct cases: (a) when the activation function is applied after the DW operation, (b) when the activation function is not applied after the DW operation, (c) when the activation function is applied after the PW operation, and (d) when the activation function is not applied after the PW operation

illustrates a blurred version of the feature map, where the contour information is noticeably disappearing, leading to a loss of fine-grained details.
Viewpoint 1: The activation function applied after the DW primarily operates on the output of a single channel, emphasizing local information and high-frequency details specific to that channel. When the activation function is removed, the feature map tends to lose its sharpness and precision, leading to a degradation in the definition of contours and structures.

3.2 Observation 2

Observing Fig. 3 (c) and (d), which respectively represent the cases with and without activation after pointwise convolution PW, we can see that both figures preserve the contour information. However, there are noticeable discrepancies in the color details between the two. While the overall structure remains intact, Figure (c), with activation, displays more accurate and refined color representation, whereas Figure (d), without activation, shows some deviations in the color distribution.
Viewpoint2: The activation function in PW convolution helps the model better understand the interactions and relationships between different channels (such as red, green, and blue). This enables the network to preserve more meaningful color information, preventing any single channel's color from becoming too dominant or overly diminished. However, in some tasks, color information may be less important. In such cases, the non-linear characteristics of the activation function could increase computational complexity without significantly improving the model's performance.

Based on the above viewpoints, we retain the precision of the details within each channel while accepting some degree of color distortion. This trade-off is made to accelerate detection.

4 Approach

In this section, we propose a **L**ess-**A**ctivation PP-**LCNet** (LALCNet) model, characterized by the removal of unnecessary activation functions. Specifically, we remove the activation function of the pointwise convolution while retaining the activation function of the depthwise convolution. This removal is not lossless and introduces new noise during the extraction process. We analyze its effects and develop dual reduction strategies aimed at enhancing detection speed. In the subsequent subsections, we first illustrate less activation operations, and introduce the dual reduction strategies, and construct and train our LALCNet model.

4.1 Less Activation

The PW operation typically used to change the number of channels or integrate information, its function is to achieve cross-channel mixing by performing convolution operations on each pixel's all channels. When the color differences in an image are amplified by an activation function after PW, it leads to the following two successive effects: 1) The activation operation typically results in more non-zero elements appearing in the feature map. The activated feature map contains richer information, including more color differences, and therefore requires more non-zero elements to be stored in memory. 2) As the number of non-zero values increases, convolutions and other operations (such as matrix multiplication) need to process more data.This increases the number of memory accesses, as the processor continuously reads and writes these non-zero values, and the computational load also increases accordingly.

Therefore, considering speed as a priority and with acceptable loss in accuracy, we remove the activation function after the PW convolution.

4.2 Reduction Strategies

When we decided to reduce the activation functions, further observations on the feature maps with reduced active functions in Fig. 4 led to the development of additional reduction strategies. As can be seen from the figure, as the detection process progresses, color distortion becomes increasingly noticeable. It is unwise to expand the computation in the later stages

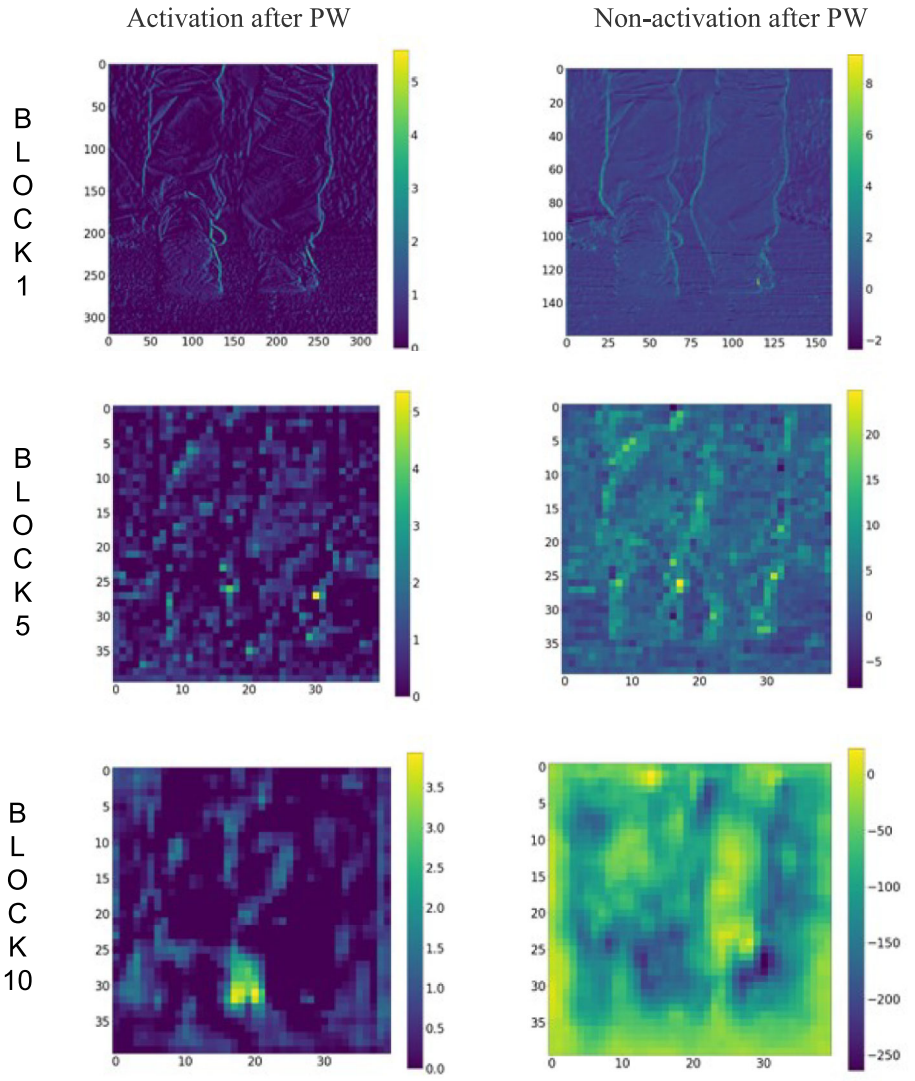


Fig. 4 The feature maps at blocks 1, 5, and 10 are shown in three rows, with the activation and non-activation after PW distributed across different columns

to improve accuracy, as this may introduce noise. We adopt further reduction strategies to further accelerate the detection process, provided the loss of accuracy is acceptable.

- **Strategy 1**: Originally, larger DW convolution kernels were used in the last 7 blocks to enhance the field and capture global context. However, after the activation function was removed following the PW operation, noise has already been introduced in the later stages. Therefore, we use 3×3 convolution kernels and do not attempt to expand the field of view.
- **Strategy 2**: By adjusting the stage ratio of the network structure from 2:2:2:7 to 2:2:6:2 and bringing the subsequent information capture forward, distortion in the later stages can be effectively reduced. The reason is that earlier stages can process and extract more important features, rather than waiting until later stages to capture information, thus reducing the risk of distortion caused by computation and noise accumulation.

Table 1 summarizes the modifications made to the PP-LCNet model.

4.3 Model Construction

Based on the dual reduction strategies, we construct LALCNet model, the model framework is shown in Fig. 5.

In accordance with Strategy 1, LALCNet reduces the convolutional kernel-size of depthwise steps in the last 7 blocks from 5×5 to 3×3, such as all of the convolutional kernel-size of depthwise steps are 3×3 from the first to the last. The unified convolution kernel size optimizes memory utilization and helps avoid the introduction of more newly noise in feature representation.

According to Strategy 2, we adjust the overall stage ratio of the network to mitigate the impact of newly generated noise on high-level feature extraction. The stage ratio refers to the distribution of the number of blocks across different stages. For the network depicted in Fig. 5, we adopt a stage ratio of 2:2:6:2, notably reducing the last stage from 7 blocks to just 2 blocks. Moreover, prior studies (Radosavovic et al. 2019, 2020) have shown that utilizing a greater number of blocks in the penultimate stage effectively balances accuracy and computational speed. Therefore, we implement 6 blocks in the second-to-last stage.

In contrast to GPU-edge computing, which is optimized for parallel computation and excels in processing tasks that can be executed simultaneously, CPU-edge computing is better suited for handling tasks that require sequential processing or have lower levels of parallelism. The LALCNet model is designed in a block-by-block manner, which aligns effectively with the sequential processing capabilities of CPUs.

Table 1 Model Optimization Strategy Comparison

Modification	Original → Updated	Rationale & Impact
PW Activation	Kept → Removed	Improves speed with acceptable accuracy loss. Activation removal increases noise.
DW-Kernel Size	Large late-stage kernels → Uniform 3×3	Prevents noise propagation in later stages, maintaining feature stability.
Stage Ratio	2:2:2:7 → 2:2:6:2	Shifts global feature capture earlier to reduce late-stage noise distortion.

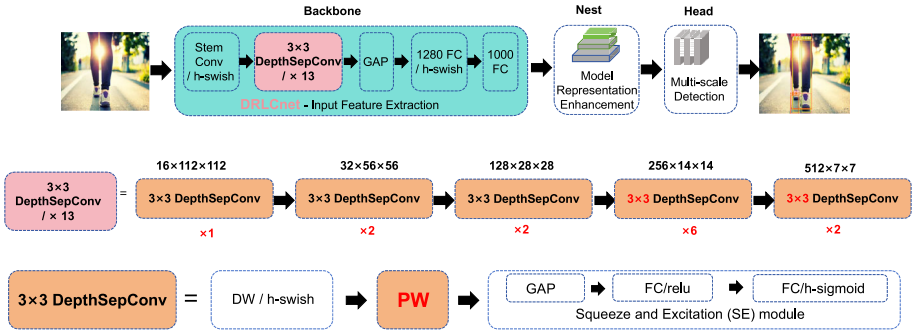


Fig. 5 Overview of LALCNet-YOLO

4.4 Model Training

To improve the robustness of the model, we replace the backbone network of YOLOv5 into LALCNet for training the model. We employ data augmentation and hyperparameter adjustment methods. The combination of these two approaches enhances the model's accuracy and generalization ability.

Data augmentation The original input data may not reach the required resolution, necessitating the diversification of training data. By generating a variety of training samples (such as image rotation, scaling, brightness adjustments, contrast modifications, etc.), we help the model better understand and adapt to different input variations. This diversity allows the model to perform more robustly in various scenarios that may arise in real-world applications. Additionally, by introducing transformations during the training process, data augmentation can reduce the model's reliance on specific training samples, thereby decreasing the risk of overfitting. The augmented data can better simulate changes in the real world, improving the model's performance on unseen data.

Hyperparameter adjustment To accommodate the substitution of the backbone network and optimize model performance, several hyperparameters are adjusted, we adopt the warmup strategy proposed by Xiong et al. (2020) to gradually adjust hyperparameters, including the learning rate, momentum, and bias learning rate, during the initial stages of training. This approach stabilizes the model by progressively transitioning to the designated learning rate. In our study, we set the batch size to 68, adjusted the learning rate to 0.01, configured the momentum parameter to 0.937, and established the weight decay factor at 0.0005. The maximum number of iterations epoch is set to be 200 through experiments.

There are two ways to determine suitable values for these parameters: (1) For iteration epoch, we conduct some parameter experiments to find suitable parameter settings (see Fig. 6). (2) For learning rate, momentum, and weight decay factor, we refer to the recommended parameter values reported in the relevant literature (Xiong et al. 2020).

We conduct experiments on the COCO-FOOT-and-LEG dataset by varying one hyperparameter at a time while keeping the other hyperparameters fixed. The precision metric is used to evaluate the performance of the proposed model. As shown in Fig. 6, the precision of the proposed model rapidly increases to 70% within the first 50 epochs. It shows that the model provides strong learning ability in the early stage and high efficiency in feature extraction. At 100-150 epochs, the precision is improved to 80%. The proposed model enters the slow convergence stage. Comprehensive trade-off is made between precision gain and training cost, we select the epoch value that produces the highest performance gain.

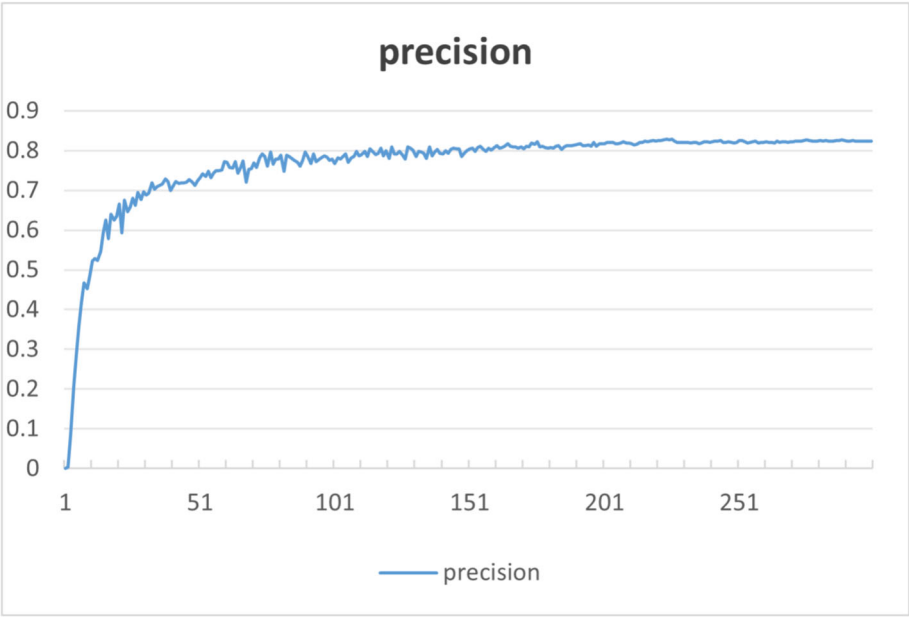


Fig. 6 Precision of different epoch values

During training, the LALCNet network progressively learns diverse scales and abstract feature representations. Our experimental results also validate that the trained model exhibits robust performance across various categories during detection.

5 Study Design

In this section, we present the details of our study, including Datesets, evaluation metrics, comparison baselines, and experiment environments.

5.1 Datasets

To evaluate the detection speed and accuracy of the LALCNet-YOLOv5, we select two representative datasets as benchmarks, including the COCO-FOOT-and-LEG and PASCAL VOC. The details of the benchmarks are described as follows.

COCO-FOOT-AND-LEG (Lin et al. 2014) is an adjunct collection derived from the MSCOCO 2017 image dataset, specifically annotated for human body parts. These annotations include categories such as 0 (body), 1 (both feet), 2 (single foot), 3 (both legs), 4 (single leg), and 5 (lower body). The dataset comprises a total of 11,056 images, encompassing the train (9,960), test (984), and validation (112) sets. This dataset is particularly suitable for robot following tasks, especially in scenarios where the robot follows a person from behind. The key objective in this task is to detect and track the person's legs and feet, enabling the robot to perform accurate following behavior. The COCO dataset contains rich annotations for feet and legs, providing diverse training data for the robot, helping it recognize and track human lower limb movements and positions in complex environments.

PASCAL VOC (Everingham et al. 2010),which is a commonly used dataset for object detection, is utilized for further validation of the LALCNet-YOLOv5's generalizability. This dataset encompasses both PASCAL VOC2007 and PASCAL VOC2012, comprising a total of 20 target categories, 16,551 training images, and 4,952 test images. The dataset can provide images from various real-world scenarios, further testing the robot's generalization ability and robustness in practical application environments.

5.2 Evaluation Metrics

In our evaluation, we employ Average Precision (AP), mean Average Precision (mAP), and Average Detection Latency (ADL) to gauge category-specific accuracy, overall accuracy, and detection speed, respectively.

AP is computed by integrating the precision-recall curve, denoted as

$$AP = \int_0^1 P(R)dR, \tag{5}$$

where P(R) encapsulates the precision-recall relationship, covering all feasible recall values from 0 to 1.

mAP serves as a crucial metric in assessing object detection models. It is derived from the AP calculated for each category detected by the model. The mAP is computed as

$$mAP = \frac{\sum_{i=1}^{N} AP_i}{N}, \tag{6}$$

where N represents the total number of assessed categories, and AP_i denotes the AP for the ith category. This computation involves averaging the AP values across all categories, providing a comprehensive evaluation of the model's object detection accuracy.

Latency signifies the average latency time required for object detection per image. To derive this metric, we select 100 images from the test dataset for detection and subsequently compute the average detection time per image. Previous works (Chen et al. 2022; Tan and Le 2019) optimize the inference speed of models based on metrics like floating point operations (FLOPs) or model sizes. However, these metrics do not correlate well with real-world latency in lightweight robots (Li et al. 2022). Hence, we measure the actual time as the benchmark metric.

5.3 Comparison Baselines

This paper introduces a novel lightweight YOLO architecture tailored for object detection on CPU platforms. In order to evaluate the efficacy of our approach, we consider four prominent lightweight object detection networks as benchmarks.

CSPDarknet is the backbone network of the YOLOv5 (Hussain 2023). It introduces the Cross Stage Partial Network (CSP) structure, which can effectively alleviate the gradient vanishing problem and improve training speed and detection accuracy to a certain extent.

ShuffleNetv2 (Zhang et al. 2018) is a CNNs architecture optimized for efficient and accurate feature extraction in computer vision tasks, particularly on resource-constrained platforms such as mobile and embedded devices.

MobileNetV2 (Sandler et al. 2018) and MobileNetV3 (Howard et al. 2019) are lightweight deep learning architectures based on CNNs, specifically designed for mobile and embedded vision applications. These models represent an improvement over the original MobileNet,

aiming to enhance performance while maintaining computational efficiency and minimizing model size.

EfficientNet (Tan and Le 2019; Zhang et al. 2024) uses a method called "compound scaling" to balance scaling across the model's width, depth, and resolution. Compared to traditional network architectures,

GhostNet (Han et al. 2020) employs a design known as the "Ghost module," which generates more feature maps without significantly increasing the computational load, thereby improving the network's efficiency.

In this study, we replaced the backbone network of the YOLO series with both the baseline networks and our proposed LALCNet. These variants serve as the foundation for the subsequent research efforts.

5.4 Experiment Environments

In order to study the suitability of the trained detection model for lightweight robots, we divided the training and detection processes into two stages and deployed them on different machines.

- In the training process, the experiments were set up with CUDA11.8 and Python3.8 and conducted on the systems equipped with AMD EPYC 7542 @3.20 GHz 32G CPUs and NVIDIA GeForce RTX 3090 24 GB GPUs.
- In the detection process, the experiments run under the Windows operating system and the PyTorch framework. The software environment is CUDA11.8 and Python3.8. The CPU is Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz and the GPU is NVIDIA GeForce RTX 2070 with Max-Q Design.

6 Performance Analysis and Discussion

Dedicated to enhance object following capability, it is imperative that the detection model prioritizes identification speed improvement in lightweight CPU-edge devices within an acceptable range of accuracy loss. To assess LALCNet, we conduct a large-scale study to seek to address the following research questions (RQ).

- **RQ1**: How does the detection speed and accuracy of LALCNet compare to different YOLO versions and baseline networks?
- **RQ2**: Does our method remain robust with only minimal loss for detecting a wide variety of objects?
- **RQ3**: Is the loss acceptable for object following tasks?
- **RQ4**: Do the followed users feel an improvement in their interaction experience?
- **RQ5**: How is the accuracy of LALCNet in different scale models?

6.1 RQ1

Setup. In exploring the first research question, LALCNet was employed as a drop-in replacement for the backbone feature extractor in the YOLO series. Its performance was compared with other baseline backbone networks. The mAP(0.5) of LALCNet and the baseline network was computed at the training end on the COCO-FOOT-AND-LEG dataset. The $Latency$ values were then recorded at the detection end to assess detection accuracy and latency.

Additionally, the number of parameters and layers of the trained model were specified to indicate its size.

Results and Analysis The results and baseline comparisons are shown in Table 2. Based on the data, we analyze the method's compatibility with the YOLO series and evaluate the detection speed of various backbone networks.

1) Our method consistently strikes a superior balance between detection speed and accuracy at different stages of the YOLO series. Our approach reduces latency by 64% (from 44.8 ms to 27.3 ms), leading to a notable improvement in detection speed. The trade-off metric mAP(0.5)/Latency for our method is 27.07, significantly surpassing YOLOv5s (18.21). For YOLOv8s, our method reduces latency from 409.49 ms to 111.73 ms, achieving a 2.66x improvement in detection speed. The trade-off metric is 6.94, far outperforming YOLOv8s (2.00). For YOLOv10s, we reduce latency from 412.27 ms to 140.71 ms, resulting in a 1.93x improvement. The trade-off metric is 5.44, significantly outperforming YOLOv10s (1.96).

2) For smaller input sizes, our method achieves a latency of 27.3 ms, significantly outperforming models like MobileNetV2-S (45.9 ms), Shufflenetv2 (35.7 ms), and MobileNet3 (32.6 ms). While the mAP (0.739) is slightly lower than MobileNetV2 (0.791), the trade-off metric mAP(0.5)/Latency for our method is 27.07, clearly surpassing MobileNetV2-S (17.23). For larger input sizes, our method shows superior real-time performance with a latency of 111.73 ms, compared to EfficientNet-B0 (742.01 ms), GhostNetV1 (555.82 ms), and MobileNetV3-L (390.17 ms). Although the mAP of our method (0.775) is slightly lower than that of EfficientNet-B0 (0.821), the latency of our method is approximately 6.64 times faster than EfficientNet-B0. The trade-off metric mAP(0.5)/Latency is 6.94, significantly surpassing EfficientNet-B0 (1.11).

Answer to RQ1.Our method demonstrates a clear advantage across various models and input sizes. By achieving substantial reductions in latency while maintaining competitive mAP values, our approach significantly improves real-time detection performance. The trade-off metric mAP(0.5)/Latency consistently outperforms existing methods, indicating a strong balance between accuracy and efficiency.

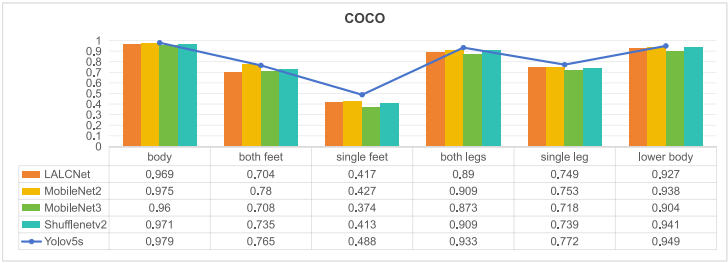
6.2 RQ2

Setup. In addressing this research question, we conducted a detailed comparative analysis of category-specific accuracy, using the AP(0.5) value as the evaluation metric. To emphasize performance loss, we set the image input size to 352×352. The COCO-FOOT-AND-LEG dataset is particularly notable for its focus on object size variations, consisting of six distinct size categories. In contrast, the PASCAL VOC dataset spans a wide range of object types, with twenty distinct categories. To ensure a comprehensive evaluation across the full spectrum of object categories, we rigorously assessed the accuracy for each category within these datasets.

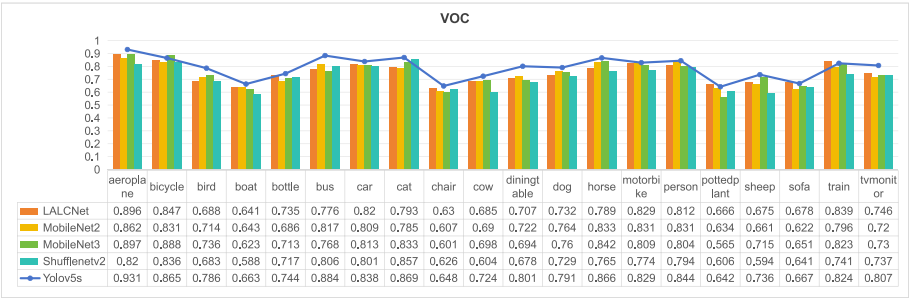
Results and Analysis. Figure 7 displays the AP(0.5) results, offering a comparative view of category-specific performance between models trained using LALCNet and baseline networks, as evaluated on the test sets of both benchmark datasets. The polygonal line denotes the accuracy value of SOTA baseline - native YOLOv5s. Within the COCO-FOOT-AND-LEG dataset, this is illustrated in the upper part of Fig. 7, which presents the AP(0.5) values for a single-class comparison between LALCNet and baseline network models. The results show a marginal decrease in accuracy for each category, regardless of object size, when compared to the native YOLOv5. Similarly, in the PASCAL VOC dataset, the accuracy loss of LALCNet-YOLOv5 also falls within a narrow range, consistent across the diverse categories.

Table 2 Results of object detection

Backbone	YOLO	Input size	Latency(ms)↓	Params(M)	Layers	mAP(0.5)↑	mAP(0.5)/Latency(s)↑
Native	YOLOv5s	352	44.8	7.03	157	0.816	18.21
MobileNetV2-S	YOLOv5s	352	45.9	2.92	276	0.791	17.23
MobileNetV3-S	YOLOv5s	352	32.6	2.14	287	0.754	23.13
Shufflenetv2	YOLOv5s	352	35.7	3.80	261	0.786	22.02
Ours	YOLOv5s	352	**27.3**	3.00	294	**0.739**	**27.07**
Native	YOLOv8s	640	409.49	11.14	129	0.819	2.00
EffcientNet-B0	YOLOv8s	640	742.01	9.99	319	0.821	1.11
GhostNetV1	YOLOv8s	640	555.82	5.9	237	0.809	1.46
MobileNetV3-L	YOLOv8s	640	390.17	288	10.43	0.805	2.06
Ours	YOLOv8s	640	**111.73**	2.13	176	**0.775**	**6.94**
Native	YOLOv10s	640	412.27	8.07	234	0.807	1.96
Ours	YOLOv10s	640	**140.71**	1.91	246	**0.766**	**5.44**



	body	both feet	single feet	both legs	single leg	lower body
LALCNet	0.969	0.704	0.417	0.89	0.749	0.927
MobileNet2	0.975	0.78	0.427	0.909	0.753	0.938
MobileNet3	0.96	0.708	0.374	0.873	0.718	0.904
Shufflenetv2	0.971	0.735	0.413	0.909	0.739	0.941
Yolov5s	0.979	0.765	0.488	0.933	0.772	0.949



	aeroplane	bicycle	bird	boat	bottle	bus	car	cat	chair	cow	diningtable	dog	horse	motorbike	person	pottedplant	sheep	sofa	train	tvmonitor
LALCNet	0.896	0.847	0.688	0.641	0.735	0.776	0.82	0.793	0.63	0.685	0.707	0.732	0.789	0.829	0.812	0.666	0.675	0.678	0.839	0.746
MobileNet2	0.862	0.831	0.714	0.643	0.686	0.817	0.809	0.785	0.607	0.69	0.722	0.764	0.833	0.831	0.831	0.634	0.661	0.622	0.796	0.72
MobileNet3	0.897	0.888	0.736	0.623	0.713	0.768	0.813	0.833	0.601	0.698	0.694	0.76	0.842	0.809	0.804	0.565	0.715	0.651	0.823	0.73
Shufflenetv2	0.82	0.836	0.683	0.588	0.717	0.806	0.801	0.857	0.626	0.604	0.678	0.729	0.765	0.774	0.794	0.606	0.594	0.641	0.741	0.737
Yolov5s	0.931	0.865	0.786	0.663	0.744	0.884	0.838	0.869	0.648	0.724	0.801	0.791	0.866	0.829	0.844	0.642	0.736	0.667	0.824	0.807

Fig. 7 Accuracy of different categories compared to baselines

Answer to RQ2. Our method LALCNet can robustly maintain a slight accuracy loss in a wide variety of objects.

6.3 RQ3

Setup. We conducted a manual comparison of categories that exhibited relatively high accuracy loss, such as the $singlefoot$ and $doublefeet$ categories in the COCO-FOOT-AND-LEG dataset, as well as the $bird$, bus and $diningtable$ categories categories in the PASCAL VOC dataset.

Discuss. We addressed the primary sources of loss as follows:

- Minor loss of confidence. This case of loss accounts for the vast majority in COCO-FOOT-AND-LEG datasets. Figure 8 shows the accuracy comparison between LALCNet-YOLOv5 and native YOLOv5s in $singlefoot$ and $doublefeet$ classes, where the box with 2 denotes $doublefeet$ class, a box with 3 denotes $singlefoot$ class. The image on the upper is the detection result of native-YOLOv5s, and the image on the lower is the detection result of this LALCNet-YOLOv5. The loss do not affect object following tasks. Because robotic systems are typically designed with a certain level of fault tolerance, they can ignore errors within a certain range to maintain stable following performance.
- Loss of non-target objects. This loss occurs in the PASCAL VOC dataset. Figure 9 a) shows the case. When the system mistakenly identifies other objects in the environment, it can more effectively maintain focus on the intended object. This ability is crucial for ensuring that the robot's following capabilities remain reliable and uninterrupted, even in complex or crowded situations.
- Confidence flipping on related objects.This loss occurs in the PASCAL VOC dataset. Figure 9 b) shows the accuracy comparison between LALCNet-YOLOv5 and native YOLOv5s in "bird" class. The loss will affect object following tasks. When confidence



Fig. 8 Comparison of detection results on the Coco-foot-and-leg dataset(upside: LALCNet, downside:YOLOv5s)

flipping on related objects, the target will be changed to from the leftmost bird to the middle bird, it will interfere with the robot's judgment, potentially leading it to select incorrect targets. Although certain scenarios, such as when multiple individuals are wearing identical shoes, may lead to target misidentification by the robot, it can reliably ensure rapid recognition and tracking of the target (whether an elderly person or a child) when they are alone. This capability is essential for the effectiveness of companionship tasks.

Answer to RQ3. Our method results in accuracy losses in most scenarios, which are acceptable for target following tasks.

6.4 RQ4

Setup We deployed four robots, two utilizing YOLOv5 and two utilizing LALCNet. Eight undergraduate students majoring in Intelligent Science were invited to interact with the

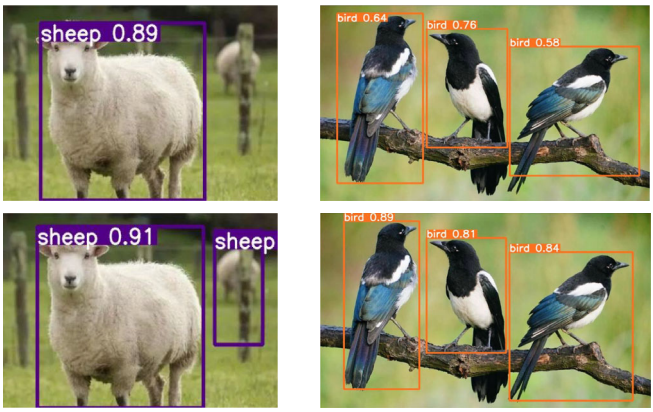


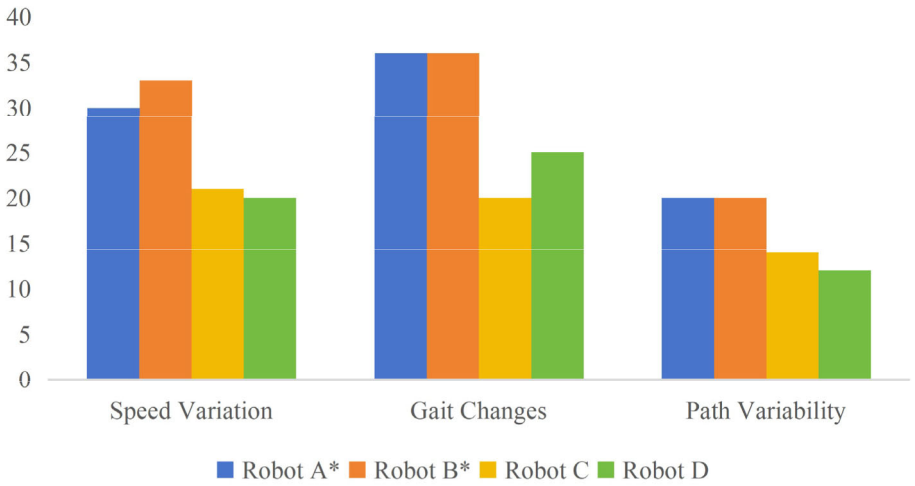
Fig. 9 Comparison of detection results on the PASCAL VOC dataset (upside: LALCNet, downside: YOLOv5s)

robots. The following evaluation metrics and testing scenarios were used to assess the robots' performance:

- **Speed Variation:** This metric evaluates the robot's ability to adjust its pace according to the user's walking speed. A score of 5 indicates that the robot can smoothly transition between slow and fast walking speeds without noticeable delays or mismatches. To assess this metric, students began by walking slowly and then gradually accelerate to a faster pace.
- **Gait Changes:** This criterion examines the robot's ability to adapt to various types of movement. It assesses the robot's flexibility in responding to different walking patterns. To test this metric, students performed dynamic movements, such as jumping.
- **Path Variability:** This metric measures the robot's ability to navigate changes in the user's path, such as turns. Lower scores are assigned if the robot deviates from the intended path or struggles to navigate turns. This metric evaluates the robot's navigational capabilities in dynamic environments. To assess this metric, students performed various turning actions.

Results and Analysis The target following ability scoring results are shown in the Fig. 10, where Robot *A* and *B* are deployed with LALCNet, and Robot *C* and *D* are deployed with YOLOv5. According to the results, the robots deployed with LALCNet show a higher user satisfaction regarding their following ability compared to those deployed with YOLOv5 in scenarios involving changes in speed, walking posture, and path. This indicates a significant improvement in their following capabilities. Specifically, the improvements in user satisfaction for changes in speed, walking posture, and path are 35%, 42%, and 35%, respectively, with the enhancement in speed changes being particularly notable.

Answer to RQ4 The LALCNet significantly enhances the following capabilities of lightweight robots, leading to increased user satisfaction during their interaction experience.



The asterisk (*) indicates that the robot's tection head uses our method

Fig. 10 Following ability satisfaction scoring situation

6.5 RQ5

Setup and Results. In this research question, we explored three scales of LALCNet in COCO-FOOT-AND-LEG dataset. The ADL and mAP(0.5) results for different scales of LALCNet are presented in Table 3.
Analysis As depicted in Table 3, compared to the SOTA baseline - CSPDark53, the speed of larger scale (scale=1) of LALCNet surpasses it by up to 38% in benchmark, while the accuracy decreased by 0.043, the speed of smaller scale (scale=0.5) of LALCNet surpasses it by up to 82%, while the accuracy decreased by 0.124. The larger the network scale, the stronger its learning and expressive capabilities, thus achieving higher accuracy. Networks of three different scales can provide users with a more flexible trade-off between speed and accuracy. Regardless, even large-scale networks can be faster than the baseline.
Answer to RQ5. Multi-scale LALCNet can provide a flexible trade-off between speed and accuracy helps users choose the appropriate network for different scenarios.

7 Ablation Study

In this ablation study, we first examined the effects of the removal of activation functions in PW steps, and then the dual reduction strategies.

7.1 Impact of Removing Activation Functions

In this experiment, we explored the impact of different activation functions used in the pointwise steps, including the removal of the activation function entirely, and the use of ReLU and H-swish. Removing the activation function resulted in a significant increase in detection speed. As shown in Table 4, compared with the PPLCNet, this change accounted for 13.75% of the speedup in the COCO-FOOT-AND-LEG dataset and 24.83% in the PASCAL VOC dataset. This indicates that the absence of activation functions allows the model to process data faster, making it particularly suitable for real-time applications. On the other side, removing the activation function also led to a performance trade-off in terms of accuracy. Specifically, removing the activation function resulted in a decrease of 0.66% and 5.47% in the mAP(0.5) value in these datasets. Removing activation functions used in the pointwise steps is a viable strategy for improving efficiency, especially when speed is a priority. The slight decrease in accuracy is an acceptable trade-off, particularly for real-time applications where processing speed is crucial.

Table 3 Comparsion of detection performance in different scales

architecture	Scale	ADL(ms)	mAP(0.5)	Params
YOLOv5	small	44.8	0.816	7M
LALCNet	0.25	24.6	0.692	2.7M
LALCNet	**0.5**	**27.3**	**0.739**	**2.9M**
LALCNet	1	32.4	0.773	3.8M

Table 4 Detection performance and latency of different configurations

Dataset			COCO-FOOT-AND-LEG			
Configurations			ADL(ms)↓	Params(M)	Layers	mAP(0.5)↑
Act(PW)	Kernel size	Stage ratio				
Hswish	5 × 5	2:2:2:7	33.1	3.32	282	0.767
Relu	5 × 5	2:2:2:7	32.4	3.32	282	0.770
–	5 × 5	2:2:2:7	29.1	3.32	254	0.762
–	3 × 3	2:2:2:7	27.9	3.30	254	0.758
–	5 × 5	2:2:6:2	27.9	3.01	294	0.747
–	3 × 3	2:2:6:2	27.3	3.00	294	0.739
Dataset			PASCAL VOC			
Configurations			ADL(ms)↓	Params(M)	Layers	mAP(0.5)↑
Act(PW)	Kernel size	Stage ratio				
Hswish	5 × 5	2:2:2:7	37.7	3.08	282	0.694
Relu	5 × 5	2:2:2:7	32.8	3.08	282	0.681
–	5 × 5	2:2:2:7	30.2	3.08	254	0.658
–	3 × 3	2:2:2:7	29.1	3.06	254	0.654
–	5 × 5	2:2:6:2	28.7	3.05	294	0.648
–	3 × 3	2:2:6:2	27.8	3.03	294	0.602

7.2 Impact of Reduction Strategies

In Table 4, when the activation function is removed, and the convolution kernel is reduced or the stage ratio is adjusted, the latency (ADL) further decreases, but the accuracy (mAP) slightly drops. Configurations with smaller convolution kernels and a 2:2:6:2 stage ratio often effectively reduces the model's computational overhead, making inference faster. This suggests that for applications with real-time or low-latency requirements, these configurations might be preferred. Although there is a slight sacrifice in accuracy, the reduction in latency can enhance real-time performance.

LALCNet demonstrates superior speed compared to other configurations across both datasets, with the removal of the activation function playing a key role in this improvement. The reduction in kernel size and adjustment of the stage ratio further enhance detection speed, under the condition that the accuracy loss is acceptable.

8 Related Work

In contrast to prior studies (Wang et al. 2023; Du et al. 2022; Liu et al. 2023; He et al. 2023), our focus lies not on prioritizing accuracy improvements under resource constraints as the primary objective, but rather on adopting a more positive perspective that *emphasizes speed enhancement within an acceptable range of accuracy loss*. We advocate for reducing the burden on the detection model while still maintaining an acceptable level of accuracy in order to achieve speed improvements, rather than pursuing accuracy enhancements at the expense of high computational costs. This approach proves particularly effective for

tracking tasks, where timely detection is crucial to prevent tracked targets from exceeding the detectable range.

8.1 Backbone Networks

In the realm of academic research, a profusion of techniques and architectural designs has surfaced to address object detection tasks. These encompass a broad spectrum of methodologies, including diverse adaptations of CNNs, such as temporal CNNS (Lea et al. 2017; Sabu and Suresh 2018; Nair et al. 2018), three-dimensional CNNs (3D CNNs) (Hou et al. 2017; Ji et al. 2012), and an array of other inventive approaches. CNN-based Object detection can be categorized into one-stage and two-stage methods. One-stage methods directly predict the location and category of objects in an image, typically offering faster speeds. Examples include YOLO (Redmon et al. 2016; Redmon and Farhadi 2017, 2018; Adarsh et al. 2020; Li et al. 2021; Shi et al. 2023), FCOS (Fully Convolutional One-Stage Detection) (Tian et al. 2019; Wang et al. 2021b, a; Tian et al. 2022), SSD (Liu et al. 2016), DSSD (Fu et al. 2017), RetinaNet (Lin et al. 2017). Two-stage methods first generate candidate regions and then classify and regress bounding boxes for these regions. Although slower, they often achieve higher accuracy. Common two-stage methods include R-CNN (Girshick et al. 2014), Faster R-CNN (Girshick 2015), Mark R-CNN (He et al. 2017).

In contrast to CNNs, Transformer-based models are better suited for handling long-range dependencies in computer vision tasks (Dosovitskiy et al. 2020; Liu et al. 2021). However, traditional Vision Transformer (ViT) suffers from quadratic computational complexity because of multi-head attention (Vaswani et al. 2017), making them less efficient than CNNs for use on mobile devices. To boost ViT efficiency, research focuses on efficient attention mechanisms (Wang et al. 2021c; Liu et al. 2021; Hatamizadeh et al. 2023; Yun and Ro 2024), model space compression (Zheng et al. 2022), knowledge distillation (Hao et al. 2022), and token reduction (Bolya et al. 2022).

Hybrid models combine the strengths of both CNNs and Transformers. CNNs are employed to extract local features, while the self-attention mechanism of Transformers processes global information. MobileViT (Mehta and Rastegari 2022) adopts a hybrid architecture, combining lightweight MobileNet blocks and multi-head self-attention (MHSA) blocks. Efficient-former (Li et al. 2022) proposes a dimension-consistent design paradigm to enhance the latency-performance boundary. RepViT (Wang et al. 2024) integrated architectural designs from efficient ViTs into mobile CNNs, leveraging structural re-parameterization (Ding et al. 2021) techniques and squeeze-and-excitation (Hu et al. 2018) modules to boost performance.

8.2 YOLO-Based Real-Time Object Detectors

YOLO involves a large amount of advanced researches (Tan et al. 2020; Ghiasi et al. 2019; Kirillov et al. 2019; Hu et al. 2021; Qiao et al. 2021; Chen et al. 2020; Oksuz et al. 2020; Zheng et al. 2020; Xu et al. 2022; Du et al. 2022; Wang et al. 2023) can be distributed in these components as following.

The backbone network plays a crucial role in extracting high-level features from input images. Recent advancements have focused on replacing traditional CNNs with variants to satisfy diversified needs. EfficientDet (Tan et al. 2020) has focused on minimizing

model parameters and computational requirements while maintaining accuracy. real-time and lightweight object detection tasks. It is characterized by its reduced model size and computational complexity compared to the full YOLOv3 network, making it suitable for deployment on resource-constrained devices such as embedded systems and mobile devices. Tiny-YOLOv3 network (Manssor et al. 2022) reduces model size and computational complexity. ShuffDet (He et al. 2023) improves the YOLOX algorithm by replacing the CSPDark53 model. Gang Liu proposes a lightweight object detection algorithm for robots with an improved YOLOv5 which the C3Ghost and GhostConv modules have been introduced into the YOLOv5 backbone (Liu et al. 2023).

The neck component serves as an intermediate component that connects the backbone and head, further processing and refining the feature maps. Typically, it includes a Feature Pyramid Network (FPN), which has many variants (Ghiasi et al. 2019; Hu et al. 2021; Qiao et al. 2021) to address scale diversity and feature fusion challenges in object detection, enhancing the model's perception of targets. The head component is the top part of the network responsible for performing specific operations for the object detection task. It aims to tackle the diversity and complexity of head structures in object detection (Dai et al. 2021), often incorporating robust loss functions (Chen et al. 2020; Oksuz et al. 2020; Zheng et al. 2020) to improve accuracy.

Unlike hybrid models (Mehta and Rastegari 2022; Li et al. 2022; Wang et al. 2024) that focus on the trade-off between accuracy and speed, our research emphasizes designing an efficient backbone network that focuses on improving the speed of target detection. In following tasks, LALCNet provides foundational support by quickly adapting to changes in the target, such as varying speeds, changes in walking posture, and turns. However, to complete target tracking, the system needs to collaborate with other modules, such as attention mechanisms (Wang et al. 2021c; Liu et al. 2021; Hatamizadeh et al. 2023; Yun and Ro 2024), residual networks (Ding et al. 2021).

9 Conclusion

The lightweight visual network LALCNet presented in this paper significantly enhances target detection speed on CPU-based edge devices while maintaining minimal accuracy loss. Experimental results demonstrate that LALCNet-YOLO achieves excellent real-time performance in complex dynamic environments (e.g., fast movements, jumping, and turning), leading to a 35%-42% improvement in user interaction satisfaction. This validates its efficiency and reliability for robot-following tasks in practical applications. Although minor accuracy fluctuations occur in highly complex environments (such as scenes with dense targets or objects of high similarity), LALCNet showcases considerable deployment potential through its multi-scale models, which support flexible trade-offs between speed and accuracy, as well as its lightweight design across various YOLO variants. The proposed method, optimized for CPU-based edge devices with limited memory, meets the stringent requirements for low latency and high responsiveness. LALCNet not only offers an efficient visual perception solution for lightweight robots but also provides valuable insights for optimizing real-time detection models in edge computing applications. LALCNet has shown good performance in tracking tasks. We can further expand its application scope. One potential direction is to apply LALCNet to other lightweight robotics scenarios, such as autonomous drones, mobile robots for indoor navigation, and robotic arms in industrial environments. By

optimizing the model for specific tasks, it can provide real-time decision-making capabilities in complex environments.

Author Contributions Jianxin Xue: Offered crucial theoretical support and guidance, engaging in in-depth discussions and analyses of research direction and methods. Husheng Chen: Responsible for designing the research and experimental setup. Yaohua Hu and Sichen Hua: Responsible for data collection and analysis. Minyu Chen and Ling-I Wu: Contributed to literature review and provided technical support for related work, validating and replicating experimental results. Xi Chang: Took charge of drafting and revising the manuscript, as well as reviewing and organizing the final submission.

Funding Not applicable

Data Availability The datasets we used for object detection can be found here (Lin et al. 2014; Everingham et al. 2010). All data were used in accordance with the terms and conditions specified by the provider.

Declarations

Ethical Approval Not applicable

Informed consent Not applicable

Conflict of interest The authors declare that they have no conflict of interest.

Clinical Trial Number in the manuscript Clinical trial number: not applicable.

References

Adarsh P, Rathi P, Kumar M (2020) Yolo v3-tiny: Object detection and recognition using one stage improved model. In: 2020 6th international conference on advanced computing and communication systems (ICACCS), IEEE, pp 687–694

Adiwahono A, Saputra V, Ng K, Gao W, Ren Q, Tan B, Chang T (2017) Human tracking and following in dynamic environment for service robots. In: TENCON 2017-2017 IEEE Region 10 Conference, IEEE, pp 3068–3073

Bolya D, Fu CY, Dai X, Zhang P, Feichtenhofer C, Hoffman J (2022) Token merging: Your vit but faster. arXiv preprint arXiv:2210.09461

Chen K, Lin W, Li J, See J, Wang J, Zou J (2020) Ap-loss for accurate one-stage object detection. IEEE Trans Pattern Anal Mach Intell 43(11):3782–3798

Chen Y, Dai X, Chen D, Liu M, Dong X, Yuan L, Liu Z (2022) Mobile-former: Bridging mobilenet and transformer. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 5270–5279

Cui C, Gao T, Wei S, Du Y, Guo R, Dong S, Lu B, Zhou Y, Lv X, Liu Q, et al. (2021) Pp-lcnet: A lightweight cpu convolutional neural network. arXiv preprint arXiv:2109.15099

Dai X, Chen Y, Xiao B, Chen D, Liu M, Yuan L, Zhang L (2021) Dynamic head: Unifying object detection heads with attentions. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 7373–7382

Ding X, Zhang X, Ma N, Han J, Ding G, Sun J (2021) Repvgg: Making vgg-style convnets great again. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 13733–13742

Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, et al. (2020) An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations

Du X, Song L, Lv Y, Qiu S (2022) A lightweight military target detection algorithm based on improved yolov5. Electronics 11(20):3263

Eisenbach M, Vorndran A, Sorge S, Gross HM (2015) User recognition for guiding and following people with a mobile robot in a clinical environment. In: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, pp 3600–3607

Everingham M, Van Gool L, Williams CK, Winn J, Zisserman A (2010) The pascal visual object classes (voc) challenge. Int J Comput Vision 88:303–338
Fu CY, Liu W, Ranga A, Tyagi A, Berg AC (2017) Dssd: Deconvolutional single shot detector. arXiv preprint arXiv:1701.06659
Ghiasi G, Lin TY, Le QV (2019) Nas-fpn: Learning scalable feature pyramid architecture for object detection. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 7036–7045
Girshick R (2015) Fast r-cnn. In: Proceedings of the IEEE international conference on computer vision, pp 1440–1448
Girshick R, Donahue J, Darrell T, Malik J (2014) Rich feature hierarchies for accurate object detection and semantic segmentation. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 580–587
Han K, Wang Y, Tian Q, Guo J, Xu C, Xu C (2020) Ghostnet: More features from cheap operations. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 1580–1589
Hao Z, Guo J, Jia D, Han K, Tang Y, Zhang C, Hu H, Wang Y (2022) Learning efficient vision transformers via fine-grained manifold distillation. Adv Neural Inf Process Syst 35:9164–9175
Hatamizadeh A, Heinrich G, Yin H, Tao A, Alvarez JM, Kautz J, Molchanov P (2023) Fastervit: Fast vision transformers with hierarchical attention. arXiv preprint arXiv:2306.06189
He K, Gkioxari G, Dollár P, Girshick R (2017) Mask r-cnn. In: Proceedings of the IEEE international conference on computer vision, pp 2961–2969
He Q, Xu A, Ye Z, Zhou W, Cai T (2023) Object detection based on lightweight yolox for autonomous driving. Sensors 23(17):7596
Hou R, Chen C, Shah M (2017) An end-to-end 3d convolutional neural network for action detection and segmentation in videos. arXiv preprint arXiv:1712.01111
Howard A, Sandler M, Chu G, Chen LC, Chen B, Tan M, Wang W, Zhu Y, Pang R, Vasudevan V, et al. (2019) Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 1314–1324
Hu J, Shen L, Sun G (2018) Squeeze-and-excitation networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 7132–7141
Hu M, Li Y, Fang L, Wang S (2021) A2-fpn: Attention aggregation based feature pyramid network for instance segmentation. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 15343–15352
Hussain M (2023) Yolo-v1 to yolo-v8, the rise of yolo and its complementary nature toward digital manufacturing and industrial defect detection. Machines 11(7):677
Islam MJ, Hong J, Sattar J (2019) Person-following by autonomous robots: A categorical overview. The International Journal of Robotics Research 38(14):1581–1618
Ji S, Xu W, Yang M, Yu K (2012) 3d convolutional neural networks for human action recognition. IEEE Trans Pattern Anal Mach Intell 35(1):221–231
Kirillov A, Girshick R, He K, Dollar P (2019) Panoptic feature pyramid networks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 6399–6408
Lea C, Flynn MD, Vidal R, Reiter A, Hager GD (2017) Temporal convolutional networks for action segmentation and detection. In: proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp 156–165
Li H, Deng L, Yang C, Liu J, Gu Z (2021) Enhanced yolo v3 tiny network for real-time ship detection from visual image. Ieee Access 9:16692–16706
Li Y, Yuan G, Wen Y, Hu J, Evangelidis G, Tulyakov S, Wang Y, Ren J (2022) Efficientformer: Vision transformers at mobilenet speed. Adv Neural Inf Process Syst 35:12934–12949
Lin TY, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick CL (2014) Microsoft coco: Common objects in context. In: Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13, Springer, pp 740–755
Lin TY, Goyal P, Girshick R, He K, Dollár P (2017) Focal loss for dense object detection. In: Proceedings of the IEEE international conference on computer vision, pp 2980–2988
Liu G, Hu Y, Chen Z, Guo J, Ni P (2023) Lightweight object detection algorithm for robots with improved yolov5. Eng Appl Artif Intell 123:106217
Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu CY, Berg AC (2016) Ssd: Single shot multibox detector. In: Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14, Springer, pp 21–37
Liu Z, Lin Y, Cao Y, Hu H, Wei Y, Zhang Z, Lin S, Guo B (2021) Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 10012–10022

Manssor SA, Sun S, Abdalmajed M, Ali S (2022) Real-time human detection in thermal infrared imaging at night using enhanced tiny-yolov3 network. Journal of Real-Time Image Processing pp 1–14
Mehta S, Rastegari M (2022) Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. In: International Conference on Learning Representations
Nair N, Thomas C, Jayagopi DB (2018) Human activity recognition using temporal convolutional network. In: Proceedings of the 5th international Workshop on Sensor-based Activity Recognition and Interaction, pp 1–8
Oksuz K, Cam BC, Akbas E, Kalkan S (2020) A ranking-based, balanced loss function unifying classification and localisation in object detection. Adv Neural Inf Process Syst 33:15534–15545
Qiao S, Chen LC, Yuille A (2021) Detectors: Detecting objects with recursive feature pyramid and switchable atrous convolution. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 10213–10224
Radosavovic I, Johnson J, Xie S, Lo WY, Dollar P (2019) On network design spaces for visual recognition. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 1882–1890
Radosavovic I, Kosaraju RP, Girshick R, He K, Dollar P (2020) Designing network design spaces. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 10428–10436
Redmon J, Farhadi A (2017) Yolo9000: better, faster, stronger. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 7263–7271
Redmon J, Farhadi A (2018) Yolov3: An incremental improvement. arXiv preprint arXiv:1804.02767
Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: Unified, real-time object detection. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 779–788
Sabu E, Suresh K (2018) Object detection from video using temporal convolutional network. In: 2018 IEEE Recent Advances in Intelligent Computational Systems (RAICS), IEEE, pp 11–15
Sandler M, Howard A, Zhu M, Zhmoginov A, Chen LC (2018) Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 4510–4520
Shi Y, Wang N, Guo X (2023) Yolov: Making still image object detectors great at video object detection. Proceedings of the AAAI Conference on Artificial Intelligence 37:2254–2262
Tan M, Le Q (2019) Efficientnet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning, PMLR, pp 6105–6114
Tan M, Pang R, Le QV (2020) Efficientdet: Scalable and efficient object detection. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 10781–10790
Tasaki R, Kitazaki M, Miura J, Terashima K (2015) Prototype design of medical round supporting robot "terapio". In: 2015 IEEE International Conference on Robotics and Automation (ICRA), IEEE, pp 829–834
Tian Z, Shen C, Chen H, He T (2019) Fcos: Fully convolutional one-stage object detection. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) pp 9627—9636
Tian Z, Chu X, Wang X, Wei X, Shen C (2022) Fully convolutional one-stage 3d object detection on lidar range images. Adv Neural Inf Process Syst 35:34899–34911
Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser, Polosukhin I (2017) Attention is all you need. Advances in neural information processing systems 30
Wang A, Chen H, Lin Z, Han J, Ding G (2024) Repvit: Revisiting mobile cnn from vit perspective. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp 15909–15920
Wang J, Song L, Li Z, Sun H, Sun J, Zheng N (2021a) End-to-end object detection with fully convolutional network. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 15849–15858
Wang T, Zhu X, Pang J, Lin D (2021b) Fcos3d: Fully convolutional one-stage monocular 3d object detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp 913–922
Wang W, Xie E, Li X, Fan DP, Song K, Liang D, Lu T, Luo P, Shao L (2021c) Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 568–578
Wang R, Duan Y, Hu M, Liu X, Li Y, Gao Q, Tong T, Tan T (2023) Lightr-yolov5: A compact rotating detector for sars-cov-2 antigen-detection rapid diagnostic test results. Displays 78:102403
Wu W, Liu H, Li L, Long Y, Wang X, Wang Z, Li J, Chang Y (2021) Application of local fully convolutional neural network combined with yolo v5 algorithm in small target detection of remote sensing image. PLoS ONE 16(10):e0259283
Xiong R, Yang Y, He D, Zheng K, Zheng S, Xing C, Zhang H, Lan Y, Wang L, Liu T (2020) On layer normalization in the transformer architecture. In: International Conference on Machine Learning, PMLR, pp 10524–10533

Xu H, Li B, Zhong F (2022) Light-yolov5: A lightweight algorithm for improved yolov5 in complex fire scenarios. Appl Sci 12(23):12312

Yun S, Ro Y (2024) Shvit: Single-head vision transformer with memory efficient macro design. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp 5756–5767

Zeng J, Xiong J, Fu X, Leng L (2020) Refpn-fcos: One-stage object detection for feature learning and accurate localization. IEEE Access 8:225052–225063

Zhang X, Zhou X, Lin M, Sun J (2018) Shufflenet: An extremely efficient convolutional neural network for mobile devices. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 6848–6856

Zhang Y, Zhan Q, Ma Z (2024) Efficientnet-eca: A lightweight network based on efficient channel attention for class-imbalanced welding defects classification. Adv Eng Inform 62:102737

Zheng C, Zhang K, Yang Z, Tan W, Xiao J, Ren Y, Pu S et al (2022) Savit: Structure-aware vision transformer pruning via collaborative optimization. Adv Neural Inf Process Syst 35:9010–9023

Zheng Z, Wang P, Liu W, Li J, Ye R, Ren D (2020) Distance-iou loss: Faster and better learning for bounding box regression. Proceedings of the AAAI conference on artificial intelligence 34:12993–13000

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.